The key points every team member should follow in RPA projects:

Bridge the Communication Gap Between Developers and BA

- Establish clear and open lines of communication between developers and BAs to ensure alignment on project requirements and objectives. Regular meetings, detailed documentation, and feedback loops are essential to address any misunderstandings or discrepancies. BAs should provide comprehensive and precise requirements, while developers should actively seek clarification and confirm their understanding to avoid misalignment and ensure that the final solution meets business needs effectively.
- After communicating with the client, it is the responsibility of both developers and Business Analysts (BAs) to review the received information thoroughly. Each team member should ensure that all client requirements and feedback are accurately understood and addressed.
- Developers should identify any technical or implementation queries arising from the client communication. Similarly, BAs should address any business or process-related questions. Both teams should collaborate to resolve these queries effectively.

Adherence to Best Practices

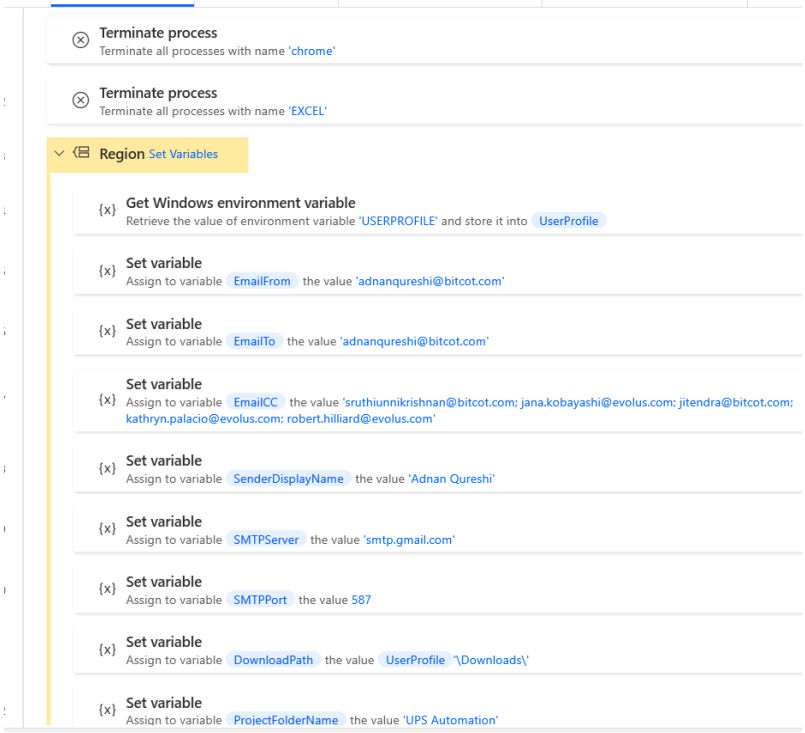
- Ensure that all RPA development follows established best practices, including modular design, reusable components, and consistent coding standards. This guarantees maintainability, scalability, and ease of understanding for all team members.

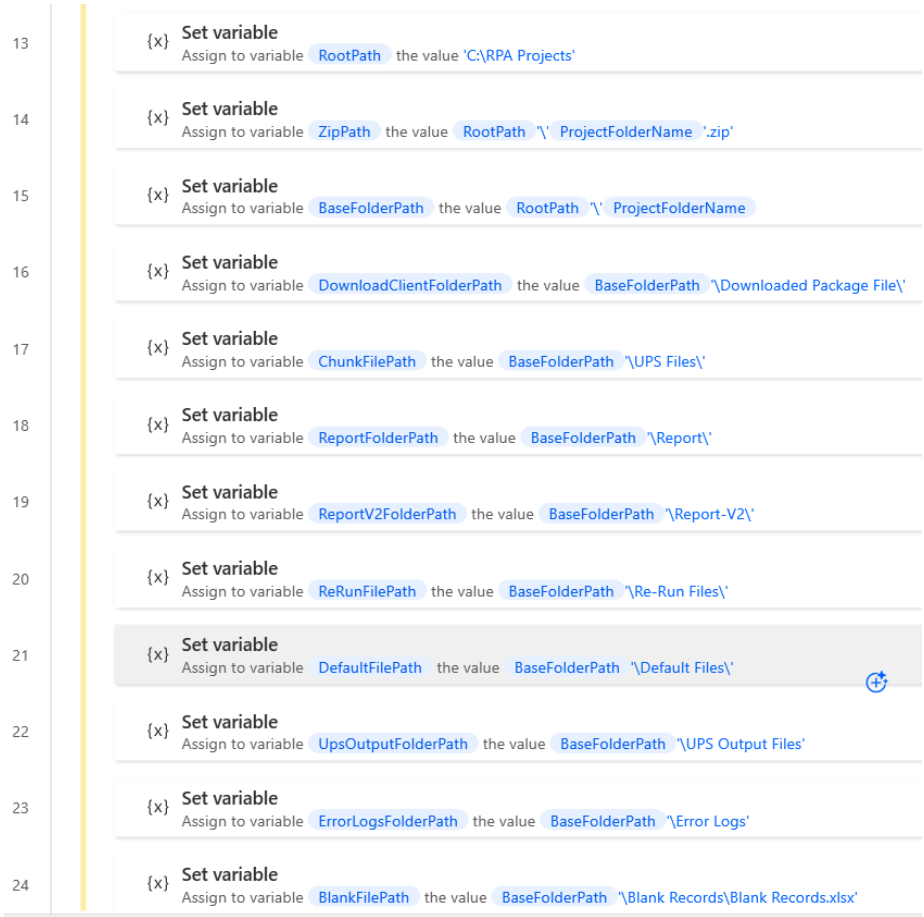
Thorough Documentation

- Document every step of the RPA process, including workflow diagrams, technical specifications, and operational procedures. Proper documentation helps in knowledge transfer, debugging, and onboarding new team members.

Use Common Variables for Repeated Elements

- Define and use common variables for frequently used elements such as folder paths, file names, and email addresses. Avoid hardcoding static values directly into your RPA workflows. This practice enhances maintainability and flexibility by allowing easy updates to these elements in one place, rather than modifying each instance individually.


Terminate process
Terminate all processes with name 'chrome'
Terminate process
Terminate all processes with name 'EXCEL'
Region Set Variables
Get Windows environment variable
Retrieve the value of environment variable 'USERPROFILE' and store it into UserProfile
Set variable
Assign to variable EmailFrom the value 'adnanqureshi@bitcot.com'
Set variable
Assign to variable EmailTo the value 'adnanqureshi@bitcot.com'
Set variable
Assign to variable EmailCC the value 'sruthiunnikrishnan@bitcot.com; jana.kobayashi@evolus.com; jitendra@bitcot.com; kathryn.palacio@evolus.com; robert.hilliard@evolus.com'
Set variable
Assign to variable SenderDisplayName the value 'Adnan Qureshi'
Set variable
Assign to variable SMTPServer the value 'smtp.gmail.com'
Set variable
Assign to variable SMTPPort the value 587
Set variable
Assign to variable DownloadPath the value UserProfile '\Downloads\'
Set variable
Assign to variable ProjectFolderName the value 'UPS Automation'


13 Set variable
Assign to variable RootPath the value 'C:\RPA Projects'
14 Set variable
Assign to variable ZipPath the value RootPath '\' ProjectFolderName '.zip'
15 Set variable
Assign to variable BaseFolderPath the value RootPath '\' ProjectFolderName
16 Set variable
Assign to variable DownloadClientFolderPath the value BaseFolderPath '\Downloaded Package File\'
17 Set variable
Assign to variable ChunkFilePath the value BaseFolderPath '\UPS Files\'
18 Set variable
Assign to variable ReportFolderPath the value BaseFolderPath '\Report\'
19 Set variable
Assign to variable ReportV2FolderPath the value BaseFolderPath '\Report-V2\'
20 Set variable
Assign to variable ReRunFilePath the value BaseFolderPath '\Re-Run Files\'
21 Set variable
Assign to variable DefaultFilePath the value BaseFolderPath '\Default Files\'
22 Set variable
Assign to variable UpsOutputFolderPath the value BaseFolderPath '\UPS Output Files'
23 Set variable
Assign to variable ErrorLogsFolderPath the value BaseFolderPath '\Error Logs'
24 Set variable
Assign to variable BlankFilePath the value BaseFolderPath '\Blank Records\Blank Records.xlsx'

22 {x} **Set variable**
Assign to variable `UpsOutputFolderPath` the value `BaseFolderPath` '\UPS Output Files'

23 {x} **Set variable**
Assign to variable `ErrorLogsFolderPath` the value `BaseFolderPath` '\Error Logs'

24 {x} **Set variable**
Assign to variable `BlankFilePath` the value `BaseFolderPath` '\Blank Records\Blank Records.xlsx'

25 {x} **Set variable**
Assign to variable `UndeliveredRecordsFormateFilePath` the value `DefaultFilePath` '\Undelivered Tracking Records Formate.xlsx'

26 {x} **Set variable**
Assign to variable `AllRecordsFormateFilePath` the value `DefaultFilePath` '\All Tracking Records Formate.xlsx'

27 {x} **Set variable**
Assign to variable `RerunReportBaseName` the value 'Rerun'

28 {x} **Set variable**
Assign to variable `CheckBlankSheet` the value 0

29 {x} **Set variable**
Assign to variable `SharePointSiteURL` the value 'https://evolusinc.sharepoint.com/sites/RPA'

30 **Run subflow** Get Date

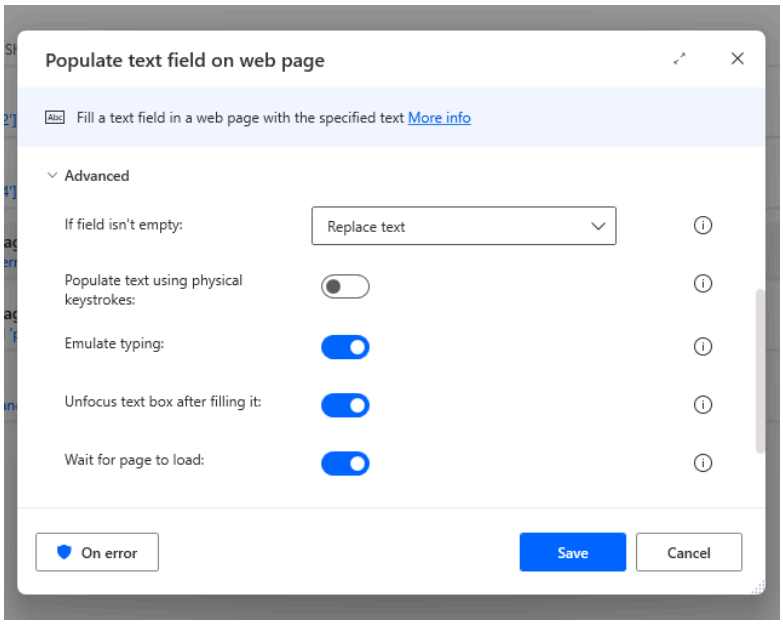
31 {x} **Set variable**
Assign to variable `SharePointFileBasePath` the value '/Shared Documents'

32 {x} **Set variable**
Assign to variable `SharePointRootPath` the value `SharePointFileBasePath` '/' `ProjectFolderName`

Don’t Use Physical Interactions with Text Fields and Button Clicks

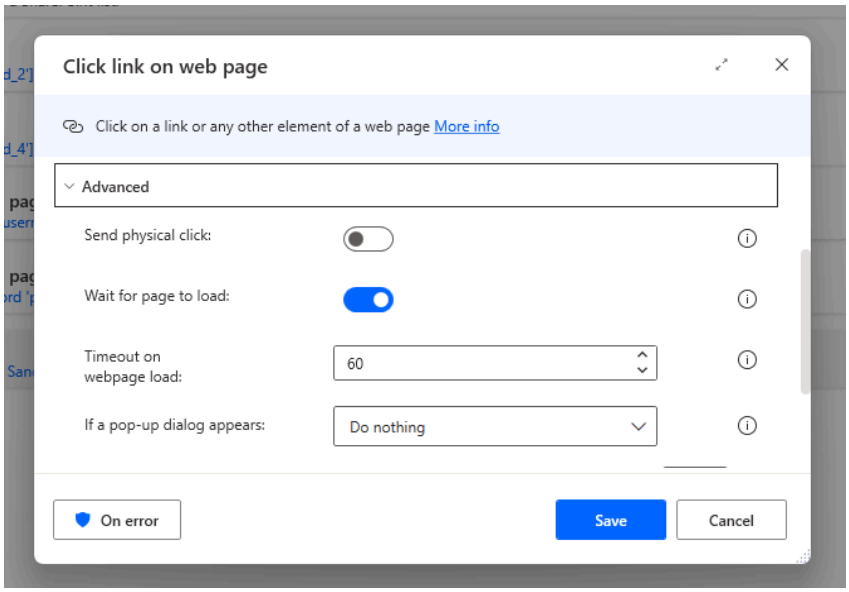
Populate Text Using Physical Keystrokes (Disabled):

- When this setting is turned off, the text is set programmatically rather than being typed out as physical keystrokes. This can make the interaction faster and more reliable.



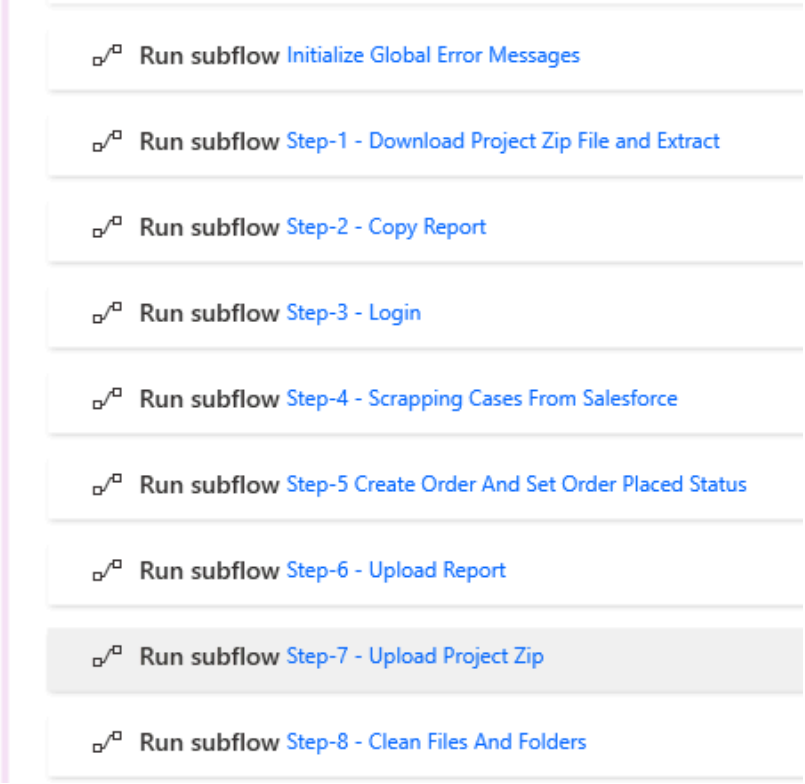
Send Physical Click (Disabled):

- The action will interact with the web page element programmatically instead of sending a physical mouse click.This can make the interaction faster and more reliable.



Utilize Sub-Flows for Modular Design

- Break down complex workflows into sub-flows to improve readability and maintainability. For example, separate common tasks like login and data insertion into distinct sub-flows. This modular approach helps in organizing the process into manageable segments, making it easier to understand, debug, and update individual components without affecting the entire workflow. Additionally, it simplifies testing by allowing you to focus on and validate each sub-flow independently.



Robust Error Handling

- Implement comprehensive error handling and logging mechanisms in all RPA bots. This ensures that exceptions are caught and logged appropriately, making it easier to identify and resolve issues during production runs.

  SQL Query Errors:

  - When an SQL query encounters an error, ensure that the whole query is captured and saved in the logs. This helps in diagnosing the issue by providing context about the problematic query.

PDF Processing Errors:

- If a PDF fails to split properly or extract text then log detailed information including:
    - The PDF name
    - The PDF page number where the error occurred
    - The path of the PDF file

  This detailed logging helps in troubleshooting issues related to PDF processing.

API Response Issues:

- Capture and save the full API response, including status codes, headers, and body content, when an API fails to respond properly. This provides insight into the nature of the issue and aids in debugging.

UI Interaction Errors:

- Log detailed error information for UI interaction issues. For instance, if a button fails to click, the log should include:
    - The element's identifier
    - The action attempted
    - Any error messages returned

Screenshots in Unattended Mode:

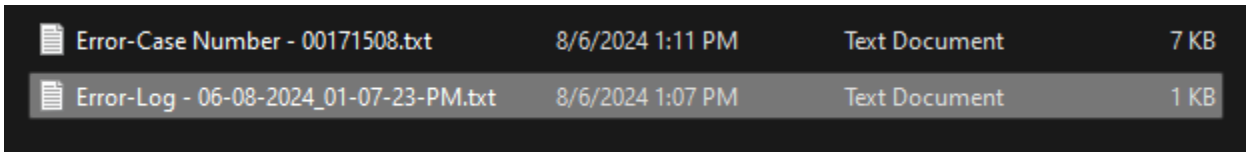
- When flow runs in unattended mode on a VM, capture and save screenshots of the error state along with the error messages. This visual evidence can be invaluable for diagnosing issues that occur during automated execution.

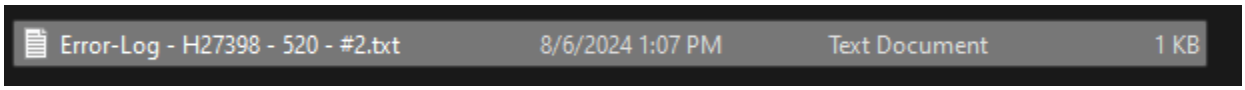
Use Unique and Descriptive Error Log File Names

- **General Errors:**
    - For general errors, include the date and time in the log file name to ensure uniqueness and track when the error occurred. For example, `Error-Log - 06-08-2024_01-07-23-PM.txt`.
- **Specific Errors:**
    - For errors related to specific elements such as PDFs, case numbers, or other identifiers, include these details in the log file name. This helps in quickly associating the error with its context. For example:
        - For a PDF-related error: `Error_Log_PDF_File Name_PDF_Page_Number.txt(Error_Log_Test_1.txt)`
        - For a case number: `Error-Case Number - Number.txt (Error-Log - Case Number - 8825654.txt)`

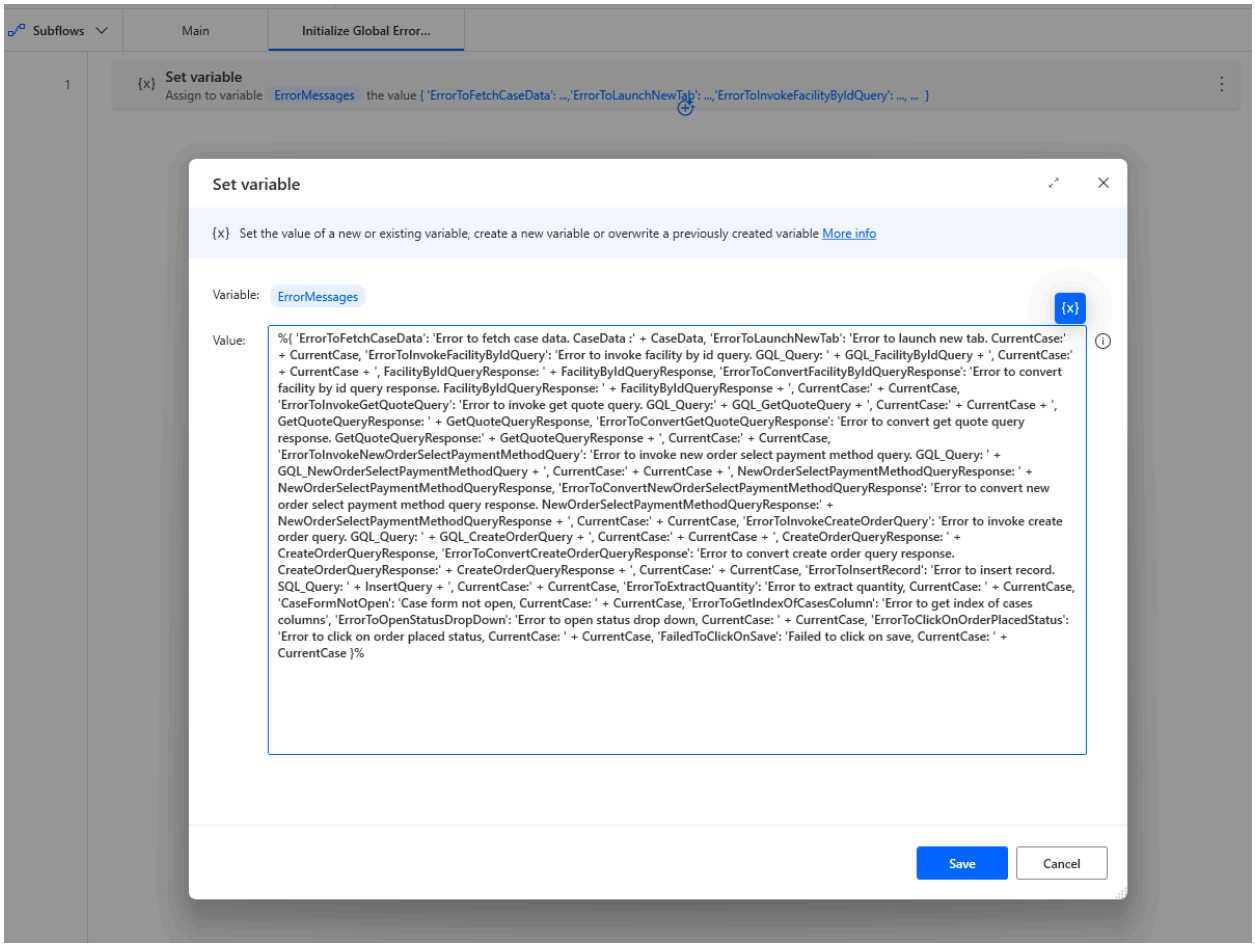
- For a UCI: `Error-Log - UCI_Number - ServiceCode - #Row_Number.txt(Error-Log - H27398 - 520 - #2.txt)`

This approach ensures that error log files are easily identifiable and related to the specific issues they document, making it simpler to locate and analyze relevant logs.

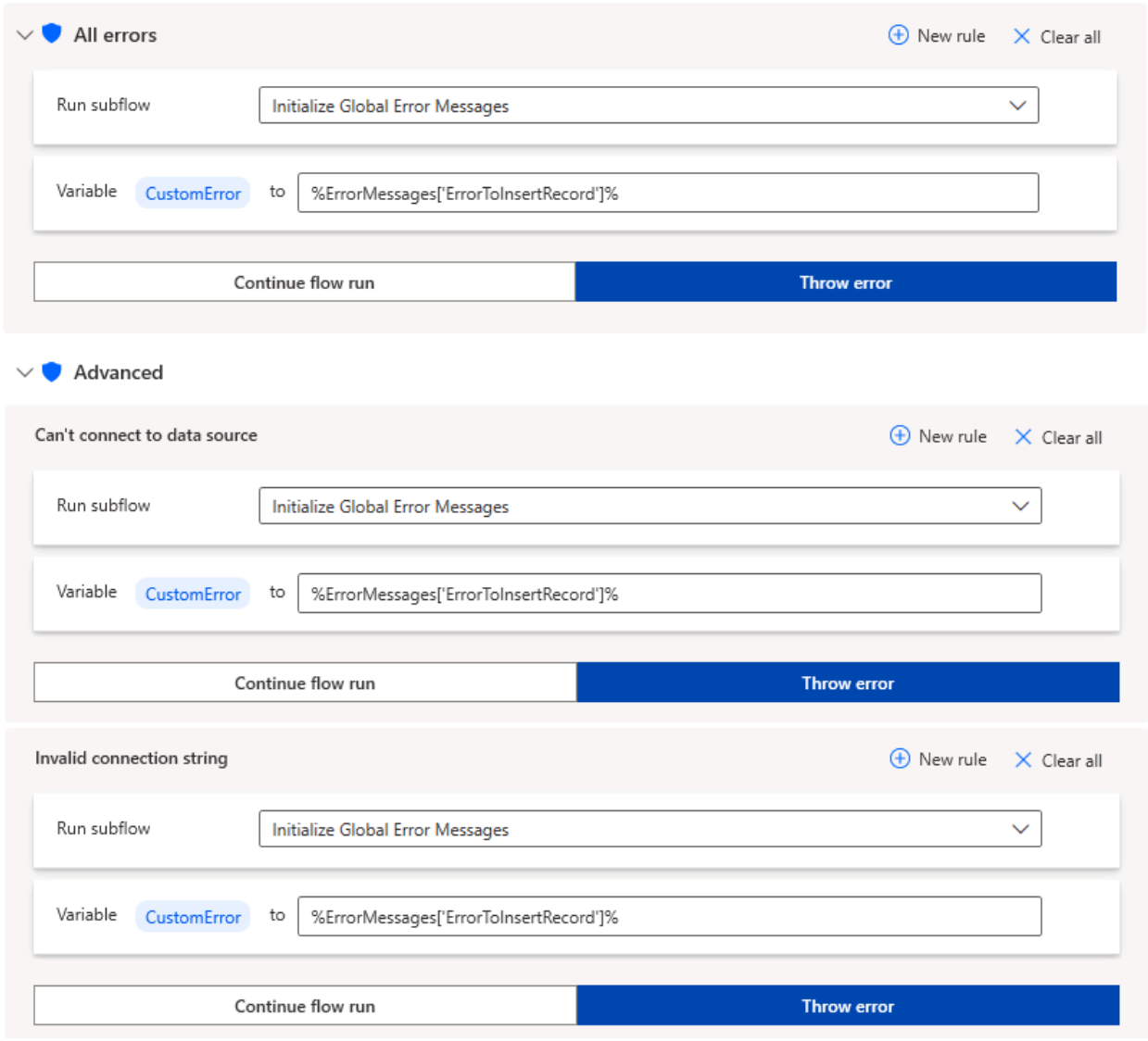




Set Global Error Messages:



Set error messages on throw error:



Save error in error logs folder

1 **Get last error**
Get the last error that occurred and store it into `LastError` and clear the error value.

2 **Convert datetime to text**
Convert datetime `CurrentDateTime` to text using format 'dd-MM-yyyy_hh-mm-tt' and store it into `FormattedDateTime`

3 **Convert datetime to text**
Convert datetime `CurrentDateTime` to text using format 'dd-MM-yyyy' and store it into `FormattedDateHour`

4 **Convert datetime to text**
Convert datetime `CurrentDateTime` to text using format 'yyyy-MM' and store it into `FormattedDateYearMonth`

5 **Create folder**
Create folder `FormattedDateHour` into `ErrorLogsFolderPath`

6 **Write text to file**
Write 'Custom Error Message: ' `CustomError` '
' `LastError` '
' `AllResponse` '
' `CurrentCase` to `ErrorLogsFolderPath` '\' `FormattedDateHour` '\' `ErrorFileName`

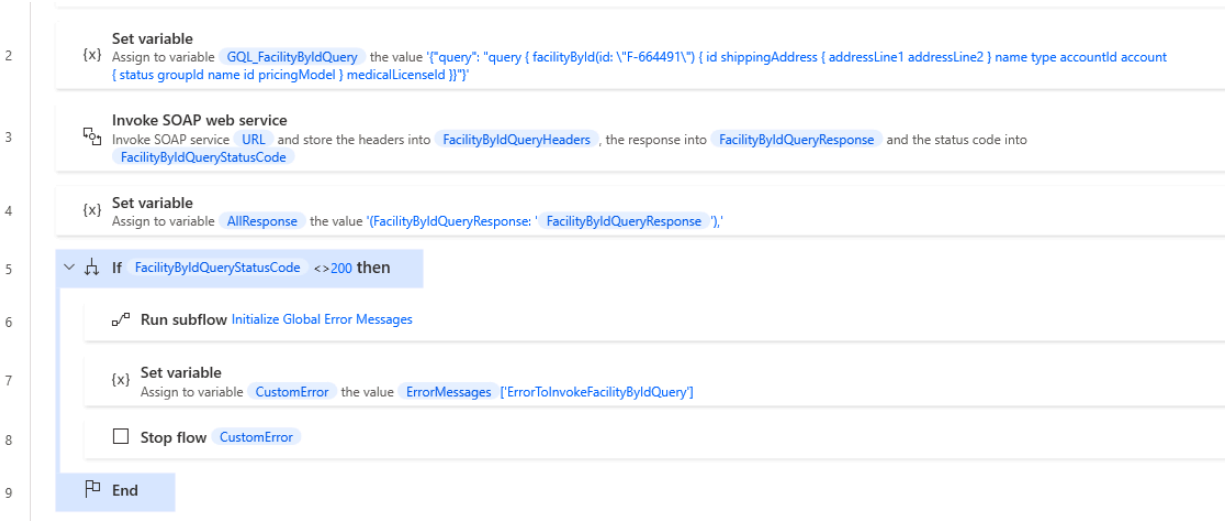
7 **Take screenshot**
Take a screenshot of the entire screen and save it to 'ScreenshotFolderPath/' `FormattedDateTime` '.png'

8 **Set variable**
Assign to variable `IsError` the value True

9 **Set variable**
Assign to variable `CustomError` the value ''

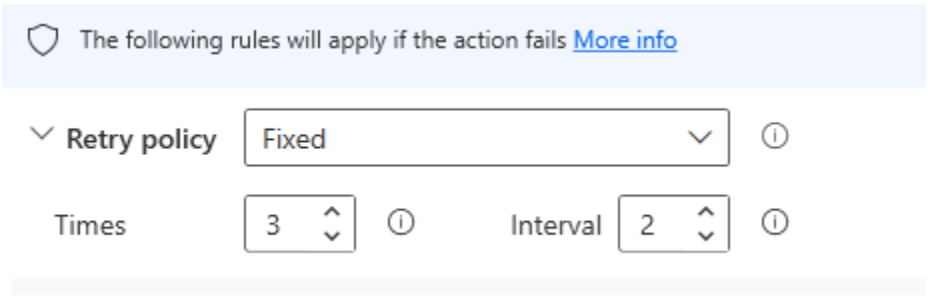
10 **Set variable**
Assign to variable `LastError` the value ''

Custom error throw



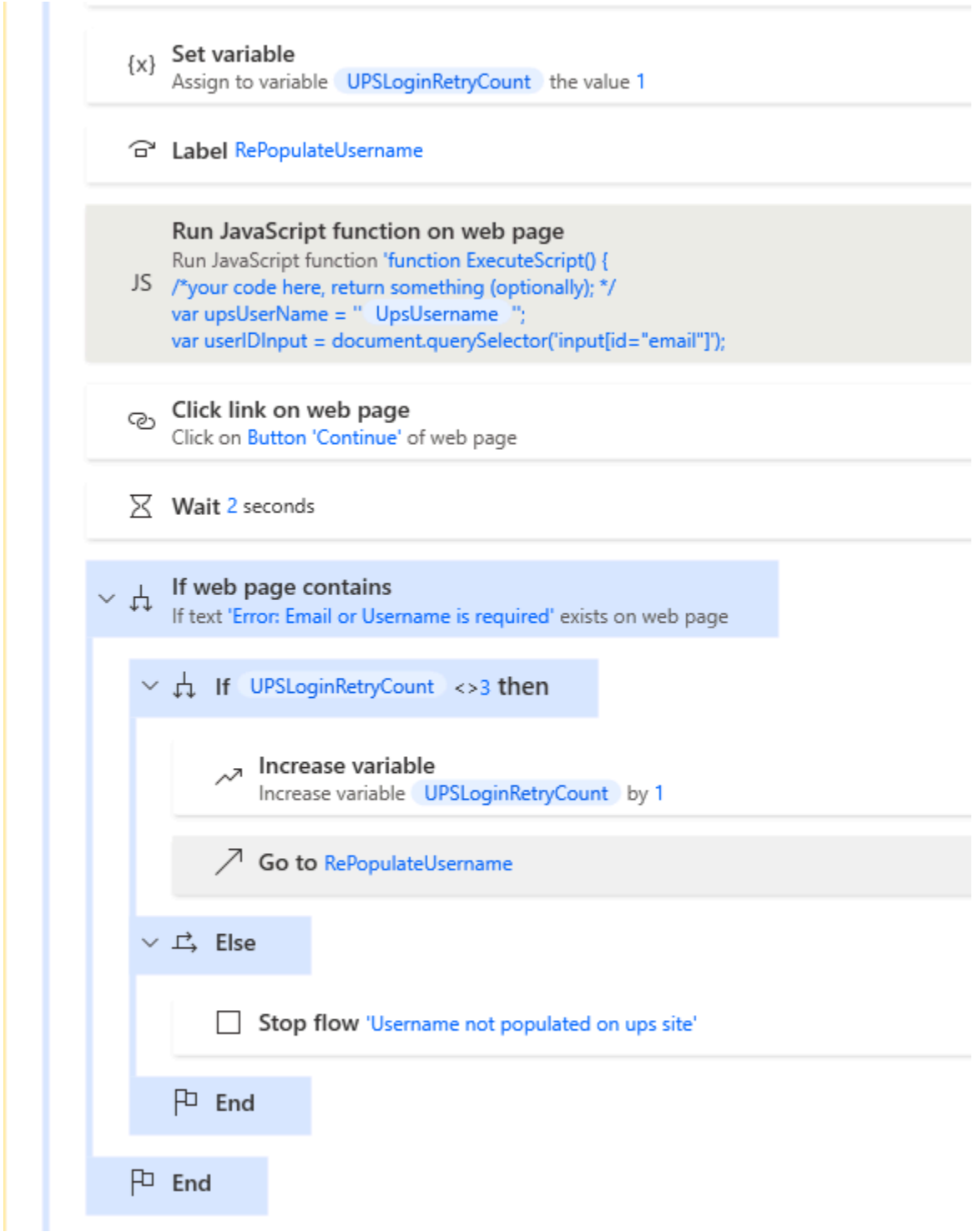
Implement Retry Mechanisms for Actions

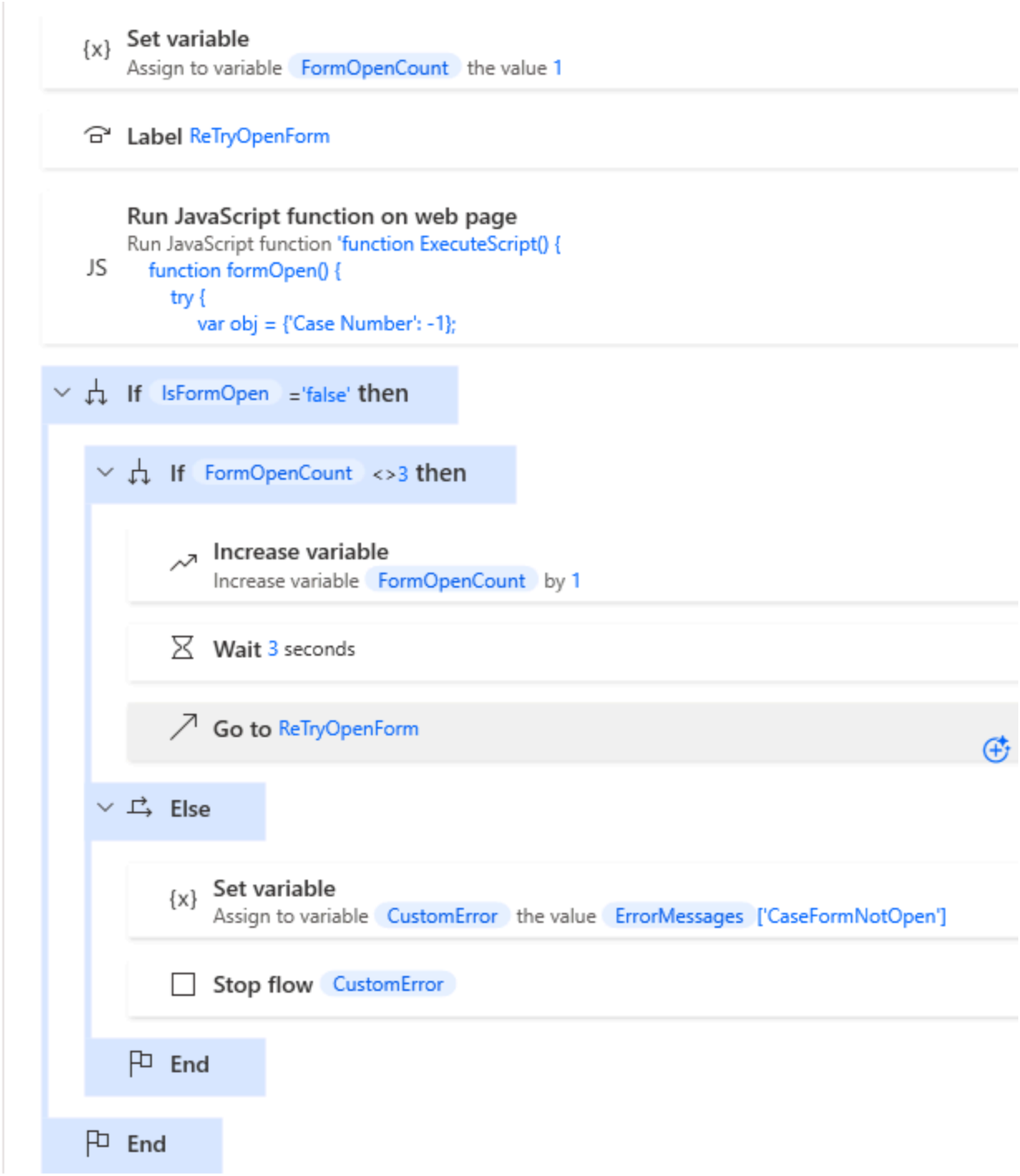
- **Universal Retry Mechanism:**
  - Every action in your RPA workflows should include a retry mechanism. Define general retry policies with parameters such as the number of attempts and wait intervals to handle transient errors and ensure successful execution.



- **Custom Retry for Login Actions:**
  - Implement specific retry logic for login actions due to their critical nature. Customize retry settings based on factors such as authentication failures or session timeouts. Ensure that the retry policy for login actions includes adequate delays and limits to manage authentication retries effectively.

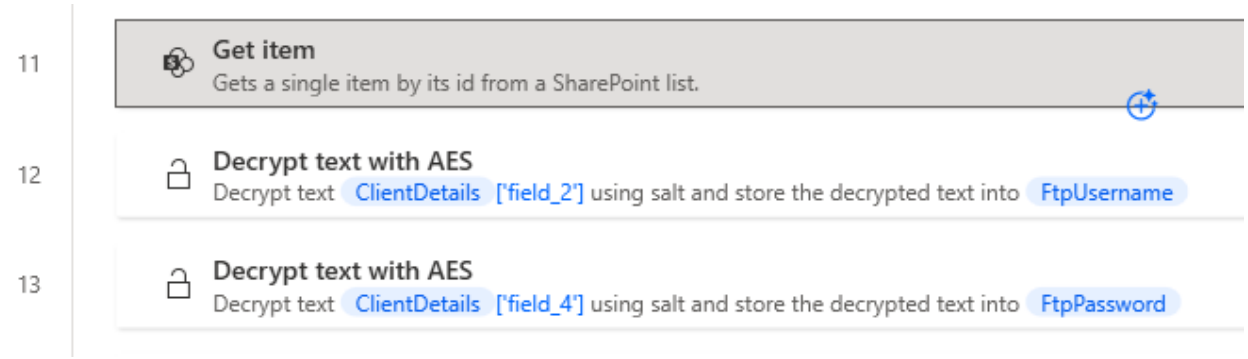
Custom retry with label




Set variable
Assign to variable FormOpenCount the value 1
Label ReTryOpenForm
Run JavaScript function on web page
Run JavaScript function 'function ExecuteScript() {
function formOpen() {
try {
var obj = {'Case Number': -1};
If IsFormOpen ='false' then
If FormOpenCount <>3 then
Increase variable
Increase variable FormOpenCount by 1
Wait 3 seconds
Go to ReTryOpenForm
Else
Set variable
Assign to variable CustomError the value ErrorMessages ['CaseFormNotOpen']
Stop flow CustomError
End
End

Security and Compliance

- Follow all security protocols and compliance requirements, including data encryption, access control, and adherence to industry regulations. This protects sensitive data and ensures that the RPA solution complies with legal standards.

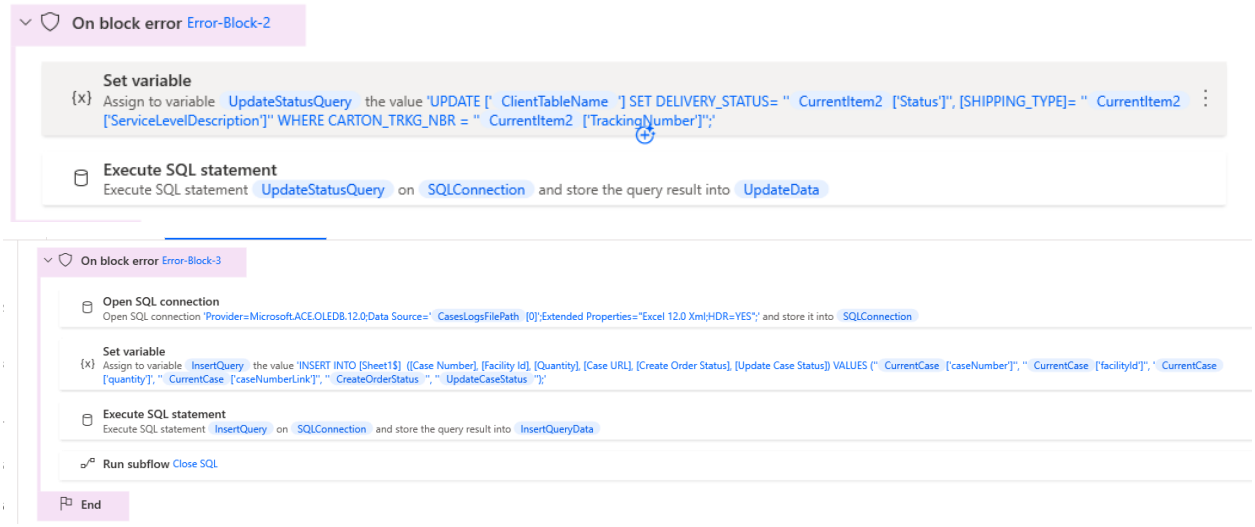


Avoid Repetitive Actions

- Avoid duplicating actions such as composing emails or performing repetitive steps within your workflows.
- Break down workflows into modular sub-flows or reusable functions for common tasks.
- Leverage variables and parameters to dynamically handle data and actions instead of hardcoding values or repeating similar steps. This approach enhances flexibility and reduces the likelihood of errors.

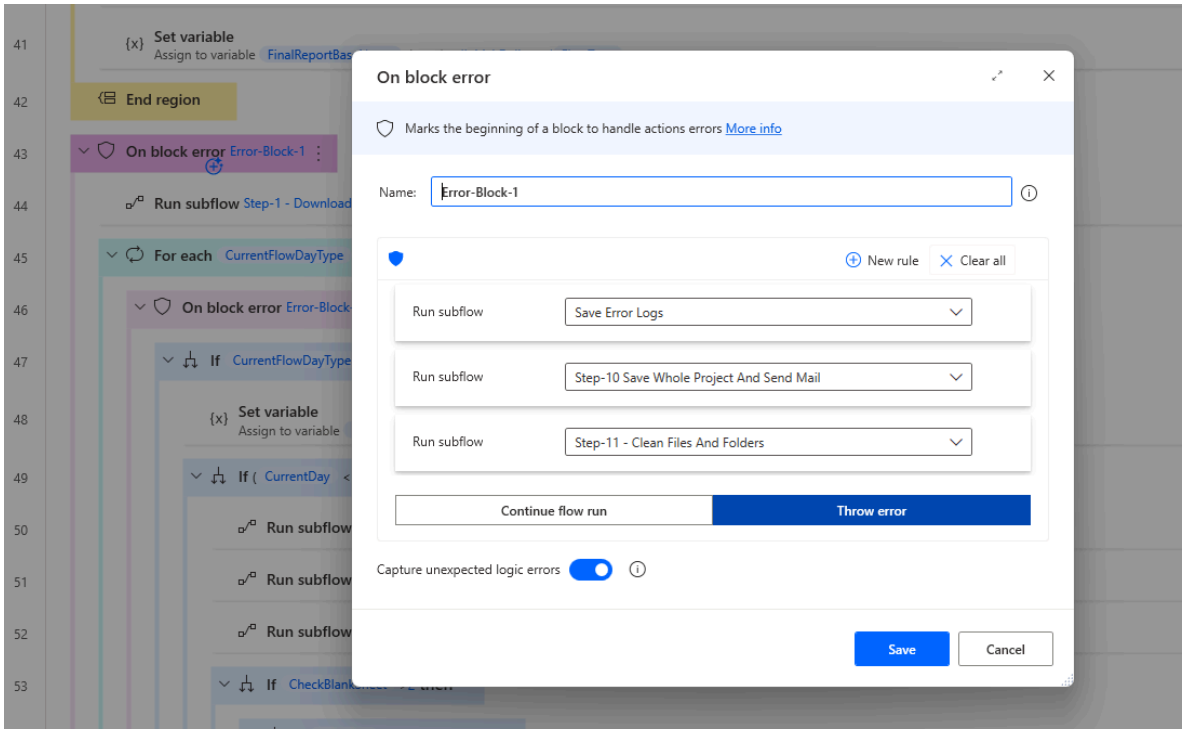
Use SQL Queries for Record Insertion and Updation Instead of VBScript

- Prefer SQL queries for record insertion as SQL is a widely understood and commonly used language. Unlike VBScript, which may not be familiar to all team members, SQL provides a more standardized and accessible approach.
- SQL queries can be easily tested and validated.
- Execute SQL queries easy to retry and save error logs as comparison VBScript.

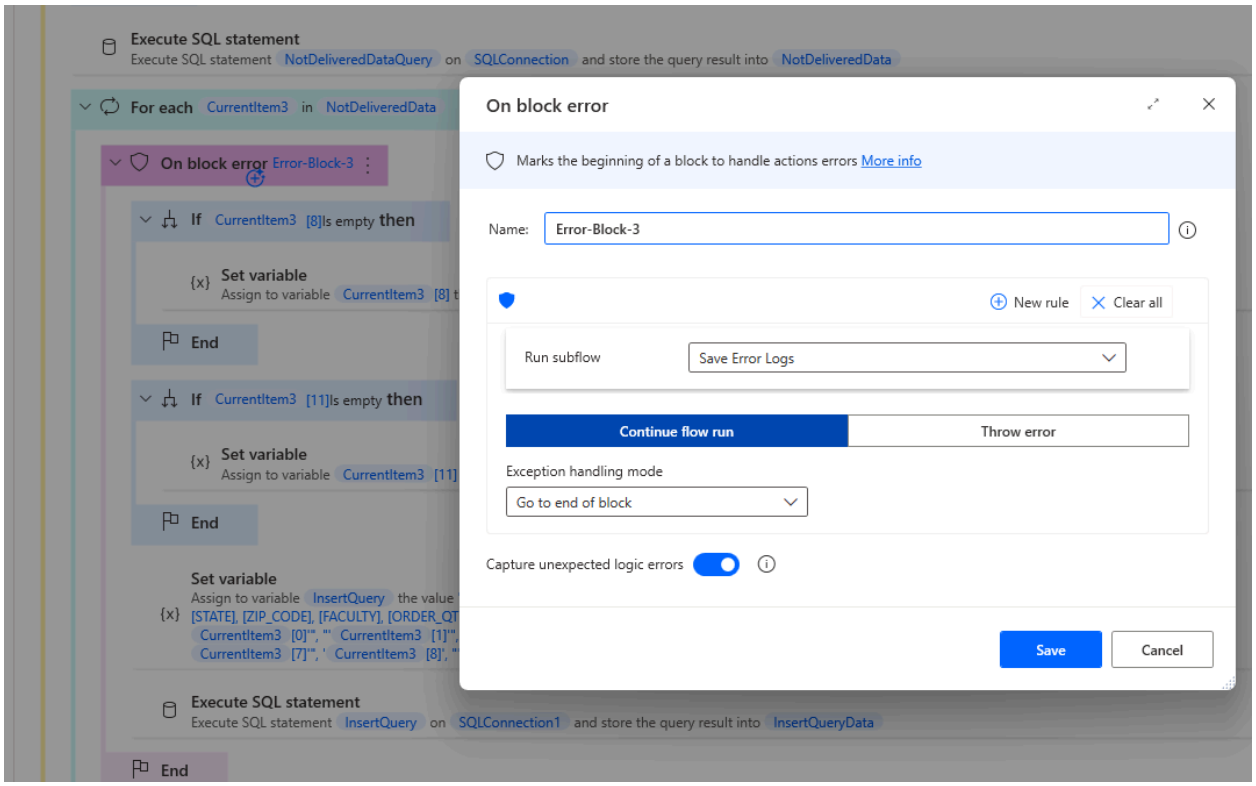


When to Stop or Continue Your Workflow

- **Stop the Workflow:**
  - **When to Stop:** If a key part of your workflow fails and can't be fixed, you should stop the workflow.
  - **Example:** If you're downloading a file from a website and it fails repeatedly after retry, stop the workflow, save error logs and send error mail.

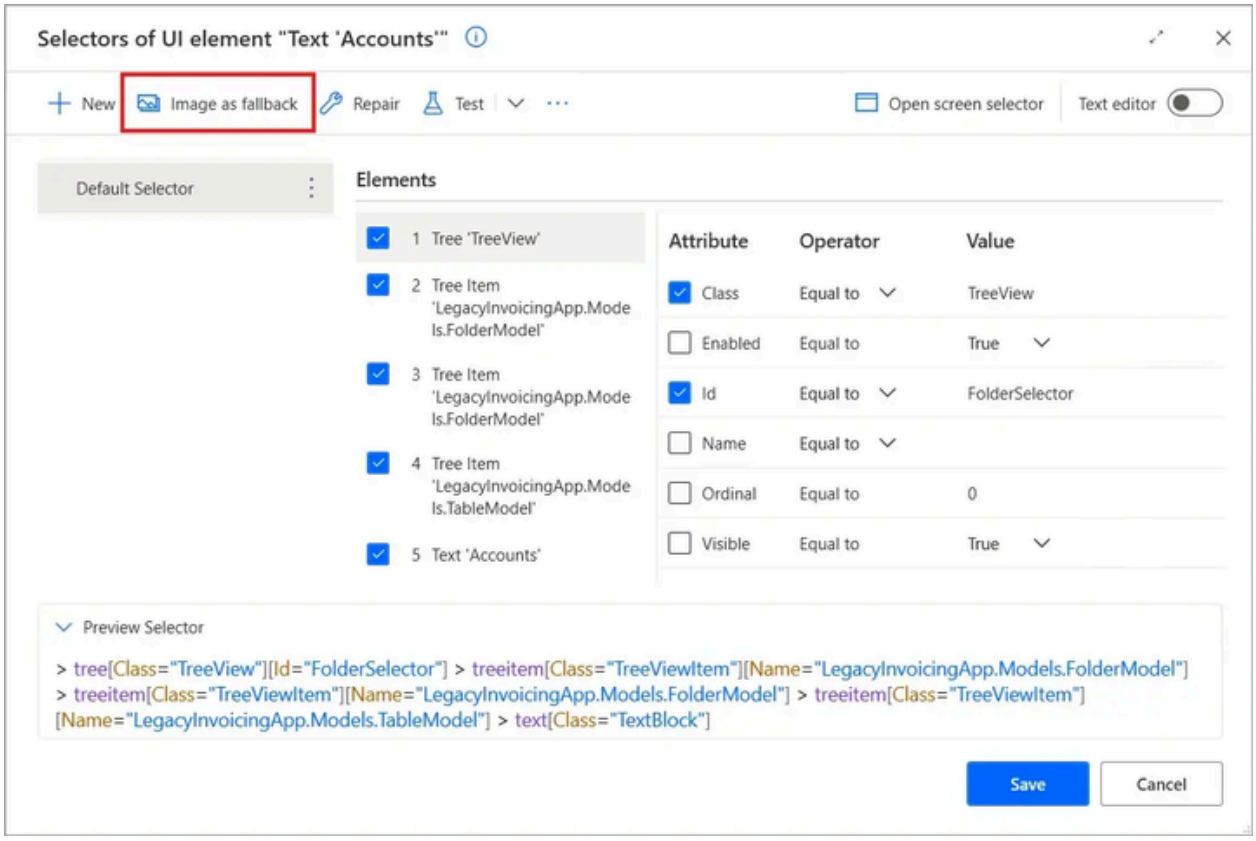


- **Continue the Workflow:**
    - **When to Continue:** If only part of the workflow fails but the rest can still proceed, let the workflow continue.
    - **Example:** If you're inserting 100 records into a database and 10 fail while 90 succeed, don't stop everything. Save 10 failed error logs.



An image-based fallback mechanism is now available for UI elements

- Power Automate for desktop now supports a robust fallback mechanism for UI elements. If the configured selectors fail to identify a UI element on the screen, users can now capture a screenshot of the element and set it as an alternative identification method. In instances where UI or browser automation actions are at risk of failing due to unallocated UI elements, the image fallback feature activates. Power Automate for desktop will then search for the UI element using the captured image and interact with it accordingly.



Send Notifications Based on Workflow Status Success or Failure

- Send a notification after the flow runs successfully or fails.

Continuous Monitoring, Testing and Validation

- Regularly monitoring, testing and validating RPA bots to ensure they perform as expected. End-to-end testing to identify and fix bugs early in the development cycle.